In [ ]:
```
import pandas as pd
dataset=pd.read_csv("insurance.csv")
dataset
```

In [ ]:
```
dataset=pd.get_dummies(dataset,drop_first=True)
dataset
```

In [ ]:
```
x=dataset[['age', 'bmi', 'children','sex_male', 'smoker_yes','region_northwest','region_southeast','region_southwest']]
y=dataset[['charges']]
print(x)
print(y)
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.20,random_state=42)
```

In [7]:
```
from sklearn.preprocessing import StandardScaler
sc=StandardScaler()
x_train=sc.fit_transform(x_train)
x_test=sc.transform(x_test)
```

In [13]:
```
from sklearn.model_selection import GridSearchCV
from sklearn.tree import DecisionTreeRegressor
param_grid={
    'criterion':['squared_error','absolute_error','poisson'],
    'max_features':['sqrt','log2'],
    'splitter':['best','random']
}
grid=GridSearchCV(DecisionTreeRegressor(),param_grid,refit=True,verbose=3,n_jobs=-1)
grid.fit(x_train,y_train)
```

```
Fitting 5 folds for each of 12 candidates, totalling 60 fits
```

Out[13]:

GridSearchCV
best_estimator_: DecisionTreeRegressor
DecisionTreeRegressor

In [ ]:
```
grid_pred=grid.predict(x_test)
print("prediction data:",grid_pred)
print("actual data:",y_test)
```

In [15]:
```
from sklearn.metrics import r2_score
result=r2_score(grid_pred,y_test)
result
```

Out[15]:

```
0.6436529900287113
```

In [16]:
```
re=grid.cv_results_
print("r_score of best parameter{}:".format(grid.best_params_),result)
```

```
r_score of best parameter{'criterion': 'absolute_error', 'max_features': 'log2', 'splitter': 'best'}: 0.6436529900287113
```

In [17]:
```
table=pd.DataFrame.from_dict(re)
table
```

Out[17]:

	mean_fit_time	std_fit_time	mean_score_time	std_score_time	param_criterion	param_max_features	param_splitter	params	split0_test_score	split1_test_score	split2_test_score	split3_test_score	split4_test_score	mean_test_score	std_test_score	rank_test_s
0	0.004193	0.000283	0.001982	0.000124	squared_error	sqrt	best	{'criterion': 'squared_error', 'max_features':...	0.661550	0.793567	0.567274	0.583522	0.586958	0.638574	0.084059	
1	0.004756	0.001195	0.003114	0.001834	squared_error	sqrt	random	{'criterion': 'squared_error', 'max_features':...	0.388494	0.676110	0.106813	0.497162	0.544377	0.442591	0.191595	
2	0.005125	0.000628	0.003688	0.002038	squared_error	log2	best	{'criterion': 'squared_error', 'max_features':...	0.624582	0.615947	0.583511	0.644970	0.666035	0.627009	0.027806	
3	0.005484	0.001408	0.002529	0.000458	squared_error	log2	random	{'criterion': 'squared_error', 'max_features':...	0.658601	0.567147	0.665981	0.520670	0.625321	0.607544	0.055708	
4	0.010098	0.002355	0.002058	0.000119	absolute_error	sqrt	best	{'criterion': 'absolute_error', 'max_features'...	0.767554	0.744037	0.242304	0.145316	0.534659	0.486774	0.254430	
5	0.008178	0.000895	0.001917	0.000226	absolute_error	sqrt	random	{'criterion': 'absolute_error', 'max_features'...	0.636885	0.500585	0.361364	0.597929	0.745607	0.568474	0.129944	
6	0.009837	0.000806	0.001926	0.000095	absolute_error	log2	best	{'criterion': 'absolute_error', 'max_features'...	0.758751	0.780678	0.482683	0.667201	0.716696	0.681202	0.106560	
7	0.009104	0.000803	0.001991	0.000168	absolute_error	log2	random	{'criterion': 'absolute_error', 'max_features'...	0.565654	0.620178	0.522871	0.629059	0.575046	0.582561	0.038684	
8	0.003637	0.000240	0.001890	0.000092	poisson	sqrt	best	{'criterion': 'poisson', 'max_features': 'sqrt...	0.554116	0.673933	0.517038	0.606474	0.519676	0.574247	0.059384	
9	0.003201	0.000239	0.001713	0.000056	poisson	sqrt	random	{'criterion': 'poisson', 'max_features': 'sqrt...	0.707482	0.413273	0.352160	0.493286	0.641372	0.521514	0.134271	
10	0.005343	0.001470	0.001922	0.000089	poisson	log2	best	{'criterion': 'poisson', 'max_features': 'log2...	0.631314	0.742153	0.519299	0.631226	0.643013	0.633401	0.070637	
11	0.003741	0.000315	0.002047	0.000263	poisson	log2	random	{'criterion': 'poisson', 'max_features': 'log2...	0.630480	0.693258	0.455184	0.441396	0.520371	0.548138	0.098636	

In [ ]: